Cost-Efficient Salient Object Segmentation

A 176K-parameter residual U-Net under a 50 ms single-core CPU budget

Prasangeet Dongre (Roll No. B23CH1033)
B.Tech Chemical Engineering, IIT Jodhpur • CSL7360 Computer Vision, Assignment 2 (Track B)

0.749	0.519	0.680	17.3 ms	0.73 MiB	0.6205
mIoU (DUTS-TE)	Boundary F1	quality Q	T_{CPU} per image	S_{disk}	CES

Abstract. We build a binary salient-object segmenter that has to label every pixel of a 256×256 photograph within 50 ms of single-thread CPU time and a 4 MiB parameter file. Our model is a three-level residual U-Net with a dilated depthwise multi-scale context block at the bottleneck and deep supervision, trained from scratch on DUTS-TR at 128×128 working resolution. On the full DUTS-TE benchmark (5,019 images) it reaches mIoU = 0.7491, Boundary F1 = 0.5192 and $Q = 0.6801$ at $T_{\text{CPU}} = 17.32$ ms with a 767,807-byte checkpoint, giving CES = 0.6205. On eleven hand-annotated photographs from our surroundings it reaches aggregate mIoU = 0.8484 and quality $Q = 0.7732$. The custom set is a small supplementary stress test, not a substitute for the official benchmark; its difficult scenes expose remaining weaknesses in contour accuracy and clutter handling.

1 Introduction

The assignment asks for a pipeline that separates the main object from the background, scored by

$$Q = 0.70\,\text{mIoU} + 0.30\,\text{BF1}, \tag{1}$$

$$\text{CES} = Q\Big(1 - 0.20\tfrac{T_{\text{CPU}}}{50\,\text{ms}} - 0.10\min\big(1, \tfrac{S_{\text{disk}}}{4\,\text{MiB}}\big)\Big), \tag{2}$$

with CES set to zero if $T_{\text{CPU}} > 50$ ms. We follow *Track B* (a small fully convolutional network written and trained from scratch). The design question is therefore twofold: how accurate can a network be, and how little of the time and storage budget does it need to spend to get there? Both penalty terms are linear, so each millisecond costs 0.4% of Q and the whole file-size term at most 10%. A model that is accurate *and* tiny wins over a bigger, slightly more accurate one.

2 Data and Preprocessing

2.1 Splits

DUTS-TR (10,553 images) is split into a training and a validation part; DUTS-TE (5,019 images) is the final benchmark. The validation set is 10% of DUTS-TR, drawn deterministically (seed 42) and stratified by foreground area: images are sorted by the fraction of mask pixels that are object, cut into strata, and sampled within each stratum, so that small and large objects appear in both parts in similar proportions.

Split	Images	Use
DUTS-TR (train)	9,498	gradient updates
DUTS-TR (val)	1,055	monitoring, checkpoint choice
DUTS-TE	5,019	final benchmark
Own photos	11	supplementary test only

Table 1. Dataset splits.

Test-set hygiene. DUTS-TE is never loaded by the training loop, never contributes gradients and is not used to pick the saved checkpoint. In full disclosure, however, DUTS-TE scores were looked at while comparing candidate designs during development, so it is not a strictly blind set. The reported figure is the single final run under the protocol below.

2.2 Preprocessing and augmentation

Training and validation use the `DUTSDataset` loader, which resizes each original DUTS-TR photograph directly to 128×128 with `cv2.INTER_AREA` and each mask to 128×128 with `cv2.INTER_NEAREST`. A mask pixel is foreground when its grayscale value is at least 128. The official DUTS-TE evaluator follows a separate path: it prepares the photograph and mask at 256×256 outside the timed region, downsamples the photograph to the model's 128×128 working resolution inside the timed region, then upsamples logits and computes metrics at 256×256. Captured photographs are also prepared at 256×256 and paired with manually annotated masks, which are thresholded at 128 to make binary targets. Inputs are converted from BGR to RGB and normalized to $[0, 1]$. During DUTS-TR training only, image and mask receive the same random horizontal flip (p=0.5), vertical flip (p=0.10) and isotropic rescaling by 0.9–1.1 (p=0.5, new border pixels are background); the image alone additionally gets brightness and per-channel gain jitter with clipping.

3 Method

3.1 Architecture

The network (Fig. 1) is a three-level residual U-Net with 175,971 trainable parameters. Each encoder block has two 3×3 convolutions with batch norm and ReLU plus a residual shortcut (projected with a 1×1 convolution when channel counts differ). Channel widths are kept very small, 16 → 24 → 40 → 56, and max pooling halves the resolution between stages.

Dilated multi-scale context. Salient objects range from a computer mouse to a whole backpack, so the bottleneck (at 1/8 of the working resolution) needs several receptive-field sizes. Three parallel branches, each a 1×1 projection to 16 channels followed by a *depthwise* 3×3 convolution with dilation 1, 2 and 4, are concatenated, projected back to 56 channels and added to the bottleneck input. Depthwise convolutions give the wider context for a fraction of the parameters and multiply–adds of a full dilated block.

Decoder and skips. The decoder upsamples, concatenates the matching encoder feature (U-Net style skip connections) and applies a residual block: $56{+}40\to40$, $40{+}24\to24$, $24{+}16\to16$. A final 1×1 convolution gives one logit per pixel. Two auxiliary 1×1 heads on the quarter- and half-resolution decoder stages exist *only during training*; at inference only the main head runs.

3.2 Loss and training

Every supervised output uses an equal mix of binary cross-entropy (with logits) and soft Dice loss,

$$\mathcal{L}_{\text{base}} = 0.5\,\mathcal{L}_{\text{BCE}} + 0.5\,\mathcal{L}_{\text{Dice}}, \tag{3}$$

and deep supervision weights the main and auxiliary outputs as

$$\mathcal{L} = 0.60\,\mathcal{L}_{\text{main}} + 0.15\,\mathcal{L}_{1/4} + 0.25\,\mathcal{L}_{1/2}. \tag{4}$$

BCE gives stable per-pixel gradients, while Dice counteracts the strong background majority that makes plain cross-entropy under-segment small objects.

We train from scratch (no pretrained weights) with AdamW for 60 epochs on a GPU, batch size 8 with 2 accumulation steps (effective 16), peak learning rate 7×10^{-4}, 3 epochs of linear warm-up followed by cosine decay to 10^{-6}, weight decay 10^{-5} (not applied to biases and normalisation parameters), gradient clipping at 1.0 and mixed precision. The checkpoint is chosen by validation quality $Q_{\text{val}} = 0.70\,\text{mIoU} + 0.30\,\text{BF1}$, which peaked at epoch 56 (Fig. 2).

4 Keeping the Pipeline Fast

The timed region starts with a 256×256 BGR photograph in memory and ends with a 256×256 array of 0s and 1s. Inside it we:

1. downscale to 128×128 with `INTER_AREA`,
2. convert BGR→RGB, scale to $[0,1]$ and build the tensor,
3. run the network,
4. bilinearly upsample the logits to 256×256,
5. apply a sigmoid, threshold at 0.5 and materialise the NumPy mask.

File I/O, resizing the original photograph to 256×256, loading the checkpoint and metric computation lie outside the timer. The main levers on cost were:

- **Half working resolution.** Running the CNN at 128×128 rather than 256×256 cuts the number of pixels processed by $4\times$; the logits are upsampled afterwards.
- **Very narrow channels** (16–56) and a context block built from depthwise convolutions.
- **No test-time extras:** no ensembling, no flip averaging, no auxiliary heads at inference.
- **Strict single-thread setup.** The four thread environment variables are set to `"1"` before importing NumPy, OpenCV or PyTorch, followed by `cv2.setNumThreads(1)` and `torch.set_num_threads(1)`; the model is warmed up once before timing.

T_{CPU} is the total `time.process_time()` over all 5,019 DUTS-TE images divided by 5,019. The storage side is equally simple: the saved file is a plain, uncompressed PyTorch state dict of 767,807 bytes (0.7322 MiB, 18.3% of the 4 MiB budget), all learned values live in it, and none are hard-coded.

Evaluation code. The official `src/evaluate.py` computes the four-connected foreground boundary and the 5×5 tolerance neighbourhood with OpenCV `filter2D`, using constant-zero borders so pixels outside the image count as background. The separate helper in `src/metrics.py` implements the corresponding boundary-neighbourhood operations with NumPy slicing. In both cases, mIoU is derived from one aggregate 2×2 confusion matrix and Boundary F1 is averaged over images.

5 Results

Hardware. All timings were measured on CPU only, with one thread, using PyTorch and OpenCV. CPU: `11th Gen Intel(R) Core(TM) i5-11400H @ 2.70 GHz`.

5.1 DUTS-TE benchmark

Table 2 summarises all three evaluation sets. On DUTS-TE the pipeline spends 17.32 ms per image (34.6% of the time budget), so both hard limits are met with room to spare and the efficiency penalty is only

$$\left(1 - 0.20\cdot\tfrac{17.32}{50} - 0.10\cdot\tfrac{767807}{4194304}\right) = 0.9124.$$

Hence CES $= 0.6801 \times 0.9124 = 0.6205$.

Background IoU (0.9106) is far higher than foreground IoU (0.5875): the aggregate confusion matrix shows 14.5M background pixels wrongly marked as object but also 11.6M object pixels missed, against 37.1M correctly found. Boundary F1 of 0.52 confirms that contours, more than regions, are the main weakness. Qualitative DUTS-TE examples are in Fig. 4: compact, high-contrast objects (butterfly, fawn) are segmented well, while large objects in clutter (the seated man) and multi-part scenes are only partially recovered.

Validation-to-test gap. Validation Q (0.853) is much higher than test Q (0.680). Part of this is the resolution mismatch noted in Table 2, and part is likely that the validation split, drawn from the same distribution as the

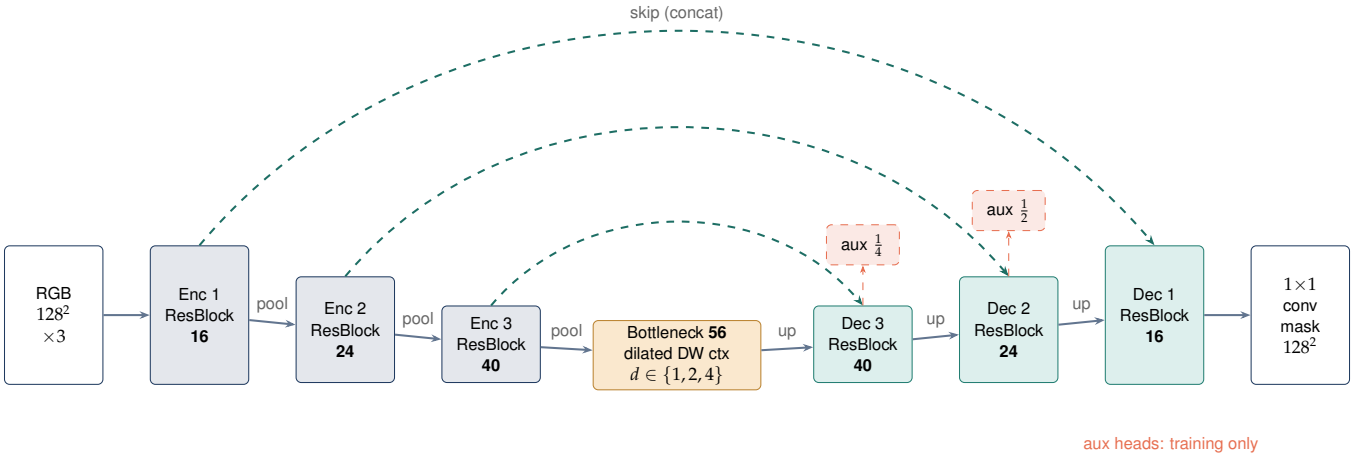


Figure 1. Architecture of the final network (175,971 parameters). Numbers in bold are channel counts. The dashed teal arcs are U-Net skip connections that concatenate encoder features into the decoder; the coral heads provide deep supervision during training and are removed at inference.

Set	Images	Res.	mIoU	BF1	Q	T_{CPU} (ms)	S_{disk} (bytes)	CES
Validation (DUTS-TR split)	1,055	128^2[a]	0.8819	0.7854	0.8529	–	767,807	–
DUTS-TE (official)	5,019	256^2	**0.7491**	**0.5192**	**0.6801**	**17.32**	**767,807**	**0.6205**
Own photos	11	256^2	0.8484	0.5978	0.7732	17.14[b]	767,807	–

Table 2. Main results. mIoU is computed from one aggregate confusion matrix; BF1 is the mean of per-image scores. [a]Validation numbers come from the 128×128 training loop and are therefore not resolution-matched with the 256×256 test numbers. [b]Mean over 220 timed predictions (20 per photograph after one warm-up); the official T_{CPU} is the DUTS-TE average. CES is reported only for DUTS-TE, the common benchmark.

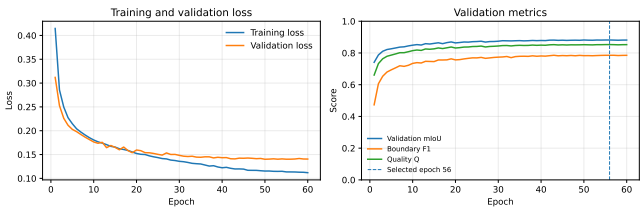


Figure 2. Training history. Left: training and validation loss. Right: validation mIoU, Boundary F1 and Q (dashed line: selected epoch 56).

training data, is easier than the official test set; the gap should not be read as a clean estimate of overfitting.

5.2 Own photographs

We retained eleven everyday photographs and annotated each target by hand on the 256×256 prepared image. The set includes a chair/clothes composite, a suitcase, two bicycles, a backpack, clogs, a controller, a mouse, a bird, a cat at a window, a kitten and a wristwatch. It deliberately includes clutter, occlusion, small foreground regions and thin structures. Fig. 3 shows the input, ground-truth and predicted overlays; Table 3 gives per-image scores. These photographs were used only for supplementary testing.

Timing over 220 predictions averaged 17.1447 ms (median 16.6661 ms, std. 1.5790 ms, min 14.5730 ms, max 22.2957 ms). These repeated timings are supplementary and do not replace the official DUTS-TE timing.

ID	Target	IoU_{fg}	mIoU	BF1
1	Chair / clothes	0.867	0.915	0.720
2	Suitcase	0.469	0.579	0.298
3	Two bicycles	0.747	0.809	0.445
4	Backpack	0.947	0.938	0.651
5	Pair of clogs	0.922	0.944	0.867
6	Controller	0.924	0.950	0.795
7	Computer mouse	0.503	0.648	0.165
8	Bird	0.930	0.960	0.867
9	Cat at window	0.570	0.768	0.435
10	Kitten	0.896	0.934	0.644
11	Wristwatch	0.897	0.940	0.689
Aggregate		0.777	0.848	0.598[c]

Table 3. Per-photo results. [c]Mean per-image BF1; mean per-image mIoU is 0.853.

6 Failure Analysis

Suitcase (ID 2): occlusion and internal detail. This is the weakest region-overlap example (foreground IoU 0.469, mIoU 0.579, BF1 0.298). The suitcase is partly covered by nearby objects; dark handles, seams and internal details interrupt the otherwise coherent surface. The prediction misses portions of the object and contains internal holes.



Figure 3. Own photographs: input, ground-truth overlay (teal) and predicted overlay (coral), with per-image mIoU. All eleven paired photographs are shown, including difficult examples.

Computer mouse (ID 7): small object and textured background. This is the weakest boundary result (mIoU 0.648, BF1 0.165). The prediction spills into parts of the high-contrast mousepad and surrounding region. At a 128×128 working resolution, a small object has few pixels available to represent each contour segment, so small displacements affect Boundary F1 strongly.

Two bicycles (ID 3): thin structures. Region overlap is moderate (mIoU 0.809), but BF1 drops to 0.445. Spokes, frame members and the overlap between the two bicycles make boundary localization difficult; the prediction can merge thin parts into a smoother region.

Cat at the window (ID 9): small foreground region. The hand-annotated foreground occupies only 5.99% of the image. Foreground IoU is 0.570, mIoU is 0.768 and BF1 is 0.435. The cat is viewed against a window with strong bars and exterior texture, making the target contour less distinct.

New high-scoring examples. The bird (ID 8) reaches mIoU 0.960 and BF1 0.867. The kitten (ID 10) reaches mIoU 0.934, and the wristwatch (ID 11) reaches mIoU 0.940. These examples demonstrate that the model can segment several compact, visually salient subjects well in this small set, but they do not establish class-level generalization.

Other examples and annotation caveats. The backpack (ID 4), clogs (ID 5) and controller (ID 6) have high region scores (mIoU 0.938–0.950), though boundary agreement still varies. ID 1 scores highly under its manually defined chair-plus-clothes composite target; this is not evidence of isolated-chair segmentation. The bicycle target also groups two objects into one region. All eleven masks are manual annotations, so boundary placement and target definitions introduce uncertainty. The custom set is selected and too small for population-level claims.

7 Discussion and Future Work

The model achieves its goal as an efficiency trade-off: 17.3 ms and 0.73 MiB leave most of both budgets unused, and CES is reduced by only 8.8% relative to Q. Since the budget is so far from binding, the most promising next steps spend some of it on quality:

- raise the working resolution (e.g. 160 or 192) and measure the gain against the 0.4%-per-millisecond cost;
- add boundary-aware supervision (e.g. an edge loss), aimed directly at the weak BF1 term;
- strengthen multi-scale fusion and add a stronger context block for large cluttered objects;
- evaluate on a larger, separately annotated hard-scene set, and take every design decision on validation data only.

Reproducibility. The repository contains the final checkpoint `final/unet_128_multiscale_v1/model_best.pt`,



Figure 4. DUTS-TE examples (input, ground truth, prediction). Good: butterfly, fawn. Failures: the seated man (large object, dark clothing blending with the sofa) and the skier (small object, spurious blob on the snow).

training and evaluation code, and the captured-photo assets: originals, 256×256 inputs, manual annotations, binary masks, predictions and review visualizations. The mask-review script is `uv run python -m src.prepare_captured_masks`; the evaluator is `uv run python -m src.evaluate_captured -checkpoint final/unet_128_multiscale_v1/model_best.pt`. The corresponding result files are `final/unet_128_multiscale_v1/results_duts-te_256.json` and `captured/evaluation/results_captured.json`.

8 Conclusion

The selected residual multi-scale U-Net meets both official resource constraints on DUTS-TE: the 256×256 evaluation reports mIoU 0.7491, Boundary F1 0.5192, and CES 0.6205 at 17.32 ms per image with a 767,807-byte checkpoint. The separate eleven-photo evaluation gives aggregate mIoU 0.8484 and quality $Q = 0.7732$, with 17.1447 ms mean CPU time across 220 timed predictions. The custom result is useful for inspecting selected real-world cases but is not a replacement for the official benchmark. Taken together, the experiments show that the model's primary remaining challenges are contour localization and robustness to clutter, occlusion, and small foreground regions rather than inference cost.